

# QuickCompare Dubai

## App Architecture & Technical Reference

*Version 1.0 | June 2026*

This document describes the current screen architecture, state management approach, data layer, and a practical roadmap for integrating real-time pricing and delivery ETAs from Dubai's leading instant delivery services.

# 1. Overview

QuickCompare is a React Native (Expo) mobile application that aggregates product listings across Dubai's major 10–15 minute delivery platforms — noon Minutes, Amazon Now, Careem Quik, and Talabat Mart — and lets users compare prices, delivery speed, and discounts before adding items to a unified cart and checking out.

The app is built with Expo Router for navigation, Zustand for global state, and a mock data layer that mirrors the structure of a real back-end API so the codebase can be promoted to production with minimal refactoring.

## 2. Screen Inventory

The app currently ships with 11 screens across two navigation stacks.

### 2.1 Bottom Tab Screens

| Screen             | Purpose                                                                                                                          |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------|
| HomeScreen         | Product grid with live search, category chips, and best-pick cards ranked by speed and price. Entry point for product discovery. |
| DealsScreen        | Ranked list of the single best discount offer per product across all in-stock services.                                          |
| OrdersScreen       | History of completed orders pulled from the Zustand orders array, with status badges and itemised lists.                         |
| AvailabilityScreen | Availability matrix showing, for every service, which products are currently in stock.                                           |
| UserScreen         | Account profile, app settings (notifications, language), and logout.                                                             |

### 2.2 Stack Screens

| Screen               | Purpose                                                                                                                           |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| SignInScreen         | Mobile-number + password auth with sign-up / log-in toggle and inline validation.                                                 |
| ProductDetailsScreen | Hero image, comparison summary (fastest / cheapest / best deal), per-service offer list with radio selector, and Add to Cart CTA. |
| CartScreen           | Enriched cart items with per-service name, line totals, +/- quantity controls, and total. Routes to PaymentScreen.                |
| PaymentScreen        | Payment method selector (Card, Apple Pay, Cash on Delivery), order summary, and checkout trigger.                                 |
| CardPaymentScreen    | Saved-card picker and new-card form with field validation before confirming payment.                                              |

|                 |                                                                                                           |
|-----------------|-----------------------------------------------------------------------------------------------------------|
| AddressesScreen | Saved address list with radio selection, add-new form, and remove (hidden when only one address remains). |
|-----------------|-----------------------------------------------------------------------------------------------------------|

## 3. Navigation Architecture

Navigation is handled by Expo Router using a hybrid stack + bottom-tab model:

- Root Stack (RootStackParamList) — wraps the whole app and owns stack screens that sit above the tabs (ProductDetails, Cart, Payment, CardPayment, Addresses).
- MainTabs (MainTabParamList) — the bottom tab navigator nested inside the root stack, containing Home, Deals, Orders, Availability, and User.
- Auth gate — SignInScreen is presented when the auth store reports no active session; the root stack switches to MainTabs on successful login.

Navigation between the tab bar and stack screens uses CompositeNavigationProp so type-safe navigation calls work from both contexts (see HomeScreen).

## 4. State Management

Two Zustand stores handle all global state:

### 4.1 useCompareStore

| Slice     | Fields & Actions                                                                                                                                       |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cart      | cartItems: { productId, serviceId, quantity }[]. Actions: addToCart, removeFromCart. Line totals are derived at render time by joining with mock data. |
| Orders    | orders[]: created by completeCheckout(), which snapshots the cart into an order record with a generated ID, timestamp, ETA, and status.                |
| Addresses | savedAddresses[], selectedAddressId. Actions: addAddress, removeAddress, selectAddress.                                                                |
| Search    | search string + setSearch — shared between HomeScreen's text input and the filtered product list.                                                      |

### 4.2 useAuthStore

| Slice    | Fields & Actions                                                                                                                      |
|----------|---------------------------------------------------------------------------------------------------------------------------------------|
| Accounts | accounts[]: { mobile, passwordHash }. Allows multiple test accounts to be registered in-session.                                      |
| Session  | currentUser: string   null. Actions: register (hashes password, pushes account), login (validates hash), logout (clears currentUser). |

## 5. Data Layer

All product and service data currently lives in two mock files. The data shapes are designed to match a real REST or GraphQL API response so the switch to live data requires only replacing the import source, not restructuring the components.

### 5.1 Core Data Types

| Type            | Key Fields                                                       |
|-----------------|------------------------------------------------------------------|
| Product         | id, name, category, unitLabel, imageSource, offers: Offer[]      |
| Offer           | serviceld, priceAed, etaMinutes, discountPct, inStock            |
| DeliveryService | id, name (noon, amazon, careem, talabat)                         |
| Order           | id, placedAt, serviceName, status, etaMinutes, totalAed, items[] |
| Address         | id, label, details                                               |

### 5.2 Compare Utility

`compareProduct(product)` in `utils/compare.ts` derives three summary values from a product's offer array that are displayed on `ProductDetailsScreen` and used for sorting on `HomeScreen`:

- fastest — the offer with the lowest `etaMinutes` among `inStock` offers.
- cheapest — the offer with the lowest `priceAed` among `inStock` offers.
- bestDeal — the offer with the highest `discountPct` among `inStock` offers.
- ranked — all in-stock offers sorted by `etaMinutes` ascending, then `priceAed` ascending.

## 6. Real-Time Prices & ETAs — Integration Roadmap

This is the most technically challenging part of the product. None of the Dubai instant delivery services (noon, Amazon Now, Careem Quik, Talabat) currently expose a public API for third-party price or ETA queries. The three viable approaches are described below, ordered from lowest to highest implementation complexity.

### 6.2 Option A — Partner / Affiliate API Access

Best for: a sustainable, ToS-compliant production product.

Risk: long lead time to negotiate, and some services may decline.

- noon has an affiliate programme (noon Affiliate) that provides product feed access with prices. Contact their partner team for API credentials.
- Amazon UAE participates in the Amazon Associates programme and the Amazon Product Advertising API (PA-API 5.0), which returns live prices and availability. ETAs are not directly exposed but can be inferred from the delivery promise shown on the product page.
- Careem and Talabat do not publicly offer partner APIs for Quik/Mart at time of writing. A business development engagement with their UAE teams is required.

- For services that provide a product feed (CSV or XML), a nightly import pipeline is sufficient for prices; ETAs still require scraping or estimation.

### 6.3 Option B — User-Contributed Data + Crowdsourcing

Best for: a community-driven MVP with zero scraping infrastructure.

Risk: data quality depends on user activity and honesty.

- When a user opens a delivery app and places an order, prompt them to confirm the price and actual delivery time after receipt.
- Aggregate reported values with a confidence score and time-decay weighting (recent reports count more).
- Display data staleness to users ("Price last confirmed 2 hours ago") to set expectations.
- Use this as a bootstrapping strategy while pursuing Option A or B in parallel.

## 7. Recommended Back-End Architecture

Regardless of which real-time option is chosen, the app should never call delivery service APIs directly from the device. A thin back end provides caching, rate-limit protection, and a stable API contract.

| Layer              | Technology & Role                                                                                                      |
|--------------------|------------------------------------------------------------------------------------------------------------------------|
| Database           | Supabase (PostgreSQL). Tables: products, services, offers (price, eta, stock, scraped_at), orders, users, addresses.   |
| Scraper workers    | Node.js or Python workers triggered by cron (every 10 min). Write results to offers table with a scraped_at timestamp. |
| API layer          | Supabase auto-generated REST + Realtime subscriptions, or a lightweight Hono/Fastify API on Railway or Render.         |
| CDN / cache        | Cloudflare in front of the API. Cache product and price responses for 5 minutes to reduce database load.               |
| Push notifications | Expo Notifications + a webhook from the back end to alert users when a watched product drops in price.                 |
| Auth               | Supabase Auth (phone OTP) replaces the current in-memory auth store. Phone OTP suits UAE users well.                   |

## 8. Features to Implement Next

### 8.1 High Priority

- Live prices and ETAs — connect the offers table to real scraper data (see Section 6).
- GPS-based ETA adjustment — use expo-location to get the user's coordinates and adjust ETA estimates based on distance from each service's nearest dark store in Dubai.
- Phone OTP authentication — replace the current mobile + password store with Supabase Auth phone OTP, which is more appropriate for UAE users and eliminates password management.
- Deep-link to service apps — the "Buy now" action should deep-link to the relevant app (noon, Amazon, Careem, Talabat) using their published URL schemes, pre-loading the product where possible.
- Real order tracking — after a deep-link or in-app purchase, poll the order status from the scraper or affiliate API and update the Orders screen.

### 8.2 Medium Priority

- Price history chart — store historical offers with timestamps and render a 7-day price sparkline on ProductDetailsScreen.
- Price drop alerts — let users watch a product and receive an Expo push notification when any service drops below a target price.
- Basket split optimisation — analyse a multi-item cart and suggest which items to order from which service to minimise total cost or total delivery time.
- Dubai area selector — dropdown or map pin to set the delivery zone, affecting which dark stores are reachable and what ETAs are realistic.

- Product image CDN — replace local `imageSource` assets with a Cloudflare Images or Supabase Storage URL so product images can be updated without an app release.

### 8.3 Lower Priority / Future

- Apple Pay integration — replace the mock Apple Pay option with a real expo-payments or Stripe Terminal implementation.
- Arabic localisation — add `i18n` support (`expo-localization` + `i18next`) for Arabic RTL layout, which is important for UAE market reach.
- Subscription / recurring orders — let users schedule a weekly basket (e.g. milk + eggs every Sunday) and auto-place it via the cheapest available service.
- Loyalty points aggregation — display loyalty balance from noon rewards and similar programmes alongside price comparisons.

## 9. Current Implementation Status

| Screen               | Status   | Notes                                                  |
|----------------------|----------|--------------------------------------------------------|
| HomeScreen           | Complete | Search, category filter, best-pick ranking, cart badge |
| ProductDetailsScreen | Complete | Service comparison, offer radio picker, add to cart    |
| CartScreen           | Complete | Qty controls, line totals, pay now routing             |
| PaymentScreen        | Complete | Method selector, order summary, COD + card routing     |
| CardPaymentScreen    | Complete | Saved cards, new card form, mock confirm               |
| AddressesScreen      | Complete | Add/remove/select address, persisted in Zustand        |
| OrdersScreen         | Complete | Order history from Zustand, status badges              |
| DealsScreen          | Complete | Top discount per product across services               |
| AvailabilityScreen   | Complete | Per-service stock matrix                               |
| UserScreen           | Complete | Profile card, settings, logout                         |
| SignInScreen         | Complete | Register + login with in-memory store                  |
| Live price scraper   | Planned  | No back end yet — all data is mock                     |
| GPS ETA adjustment   | Planned  | expo-location not yet integrated                       |
| Push notifications   | Planned  | expo-notifications not yet integrated                  |
| Deep-link to apps    | Planned  | URL scheme mapping not yet implemented                 |
| Phone OTP auth       | Planned  | Supabase Auth integration pending                      |

## 10. Known Issues & Improvements

- Cart is not service-locked: currently a user can add items from different services to the same cart. Each item stores its own serviceld, but the checkout flow treats all items as belonging to one order. Consider adding a service-lock confirmation dialog (as described in the earlier generated scaffold) to avoid mixed-service checkout edge cases.
- Password storage: the current auth store stores a password hash in-memory (not persisted). On app restart the user is always signed out. Replace with Supabase Auth.
- Image assets are bundled locally: this increases the APK size. Migrate product images to a CDN URL and load them with a caching library such as expo-image.
- No loading or error states: screens render data synchronously from mock arrays. When live API calls are added, all data-fetching screens need loading skeletons and error boundaries.
- OrdersScreen always shows mock statuses: the status field on orders is set at checkout time and never updated. A polling or WebSocket mechanism is needed for live status.
- AvailabilityScreen does not respond to selected address: stock availability can vary by dark store. Once GPS integration is live, this screen should filter by the nearest fulfilment centre.

## 11. Tech Stack Summary

| Concern            | Current / Recommended                                         |
|--------------------|---------------------------------------------------------------|
| Framework          | React Native + Expo SDK 51                                    |
| Navigation         | Expo Router (hybrid stack + bottom tabs)                      |
| State              | Zustand (compare store + auth store)                          |
| Styling            | React Native StyleSheet with a shared theme object            |
| Data (current)     | Static mock files (data/mockData.ts, data/mockUserData.ts)    |
| Data (target)      | Supabase REST + Realtime, queried via react-query             |
| Auth (current)     | In-memory Zustand store with basic password hashing           |
| Auth (target)      | Supabase Auth — phone OTP                                     |
| Images             | Local assets (target: Cloudflare Images / Supabase Storage)   |
| Scraping           | Not yet built. Target: Node.js workers on Railway / Render    |
| Push notifications | Not yet built. Target: expo-notifications + Supabase webhooks |
| Location           | Not yet built. Target: expo-location for ETA adjustment       |